

Antisèche TAS 8-14

Analyse statique par interprétation abstraite - synthèse des résumés ultra courts

Objectif	Contenu
Réunion eBPF / MOPSA	Retenir le squelette concret -> abstrait, les boucles, les produits de domaines, les zones, les pointeurs et la méthode industrielle.
Lecture prioritaire	Cours 12 + 13 + 10. Cours 8-9 pour les bases, 11 pour la précision modulaire, 14 pour la méthodologie.
Angle eBPF	Tags + régions mémoire + offsets + zones + suivi de stack, avec join/widening pour éviter l'explosion de chemins.

Fil conducteur

Cours 8 : sémantique concrète
Cours 9 : abstraction sûre
Cours 10 : boucles, invariants, widening
Cours 11 : produits réduits, disjonctions
Cours 12 : domaines relationnels, zones
Cours 13 : pointeurs, mémoire, tableaux
Cours 14 : analyse industrielle spécialisée

Carte de lecture

Pour eBPF, ne lis pas tout au même niveau :

Cours	Importance eBPF	Lecture
8	Base sémantique concrète	Diagonale
9	Abstraction, sûreté, fausses alarmes	Ciblée
10	Boucles, point fixe, widening	Important
11	Combinaison de domaines, path sensitivity	Diagonale utile
12	Zones / octogones / polyèdres	Priorité max
13	Pointeurs / mémoire / stack	Priorité max
14	Méthode industrielle spécialisée	Diagonale finale

Phrase globale à retenir

L'analyse statique par interprétation abstraite consiste à définir une sémantique concrète exacte, puis une sémantique abstraite calculable qui sur-approxime tous les comportements. La difficulté est de choisir des domaines assez précis pour éviter les fausses alarmes, mais assez efficaces pour terminer.

● Cours 8 - Sémantique concrète

Base concrète : définir mathématiquement ce que fait un programme avant de l'approximer.

Résumé ultra court

Cours 8 = base concrète.

Analyse statique :
inférer automatiquement des propriétés sans exécuter.

Sémantique concrète :
définition mathématique exacte du comportement.

Langage jouet :
affectation, séquence, if, while, assert, halt, skip.

rand :
non-déterminisme, modélise entrées inconnues.

Environnement :
 $\rho : V \rightarrow Z$, état mémoire.

$E[e]$:
expression $\rightarrow$ ensemble de valeurs possibles.

$C[c]$:
condition $\rightarrow$ filtre sur ensembles d'états.

$S[s]$:
instruction $\rightarrow$ transformation d'ensembles d'états.

Boucle :
invariant $I = \text{lfp } F$
 $F(X) = R \cup S[s](C[c]X)$

Ordres / treillis :
outils pour garantir l'existence des points fixes.

Limite :
concret exact mais non calculable en général.
Les cours suivants construisent des abstractions calculables.

Lien eBPF / réunion

Dans le papier eBPF, la section de sémantique concrète joue le même rôle : définir le mini-langage, les états, les erreurs, puis prouver que l'analyse abstraite sur-approxime cette sémantique.

● Cours 9 - Abstractions de sémantiques

Passer du concret non calculable à une approximation calculable et sûre.

Résumé ultra court

Cours 9 = abstraction de sémantiques.

On remplace $P(E)$, trop concret/non calculable,
par $E\# = V \rightarrow D\#$.

$D\#$ peut être :
signes
constantes
intervalles

γ : donne le sens concret d'un élément abstrait.
 α : donne la meilleure abstraction d'un concret.

Correspondance de Galois :
 $\alpha(c) \sqsubseteq a \Leftrightarrow c \leq \gamma(a)$

Sûreté :
toute propriété prouvée dans l'abstrait est vraie dans le concret.

Fausse alarme :
possible si l'abstraction est trop grossière.

Opérateur abstrait optimal :
 $F\# = \alpha \circ F \circ \gamma$

Mais en pratique :
on accepte des opérateurs sûrs non optimaux.

Analyse non-relationnelle :
une abstraction par variable.
Simple, efficace, mais oublie les relations entre variables.

Constantes :
exact si valeur unique, sinon $\top$.

Intervalles :
bons pour bornes, mais ne prouvent pas les relations type $x \leq y$.

Limite :
pour les boucles il faudra le cours 10.

Lien eBPF / réunion

La section 5 du papier eBPF fait exactement ce passage : états concrets avec registres/mémoire/tags, puis état abstrait calculable avec tags abstraits, domaine numérique et informations mémoire.

● Cours 10 - Analyse de boucles

Calculer des invariants de boucle avec point fixe, widening et narrowing.

Résumé ultra court

Cours 10 = analyse des boucles.

Boucle :
chercher invariant I.

Équation :
 $I = F(I)$
 $F(X) = R \cup S[[s]](C[[c]]X)$

Sortie :
 $S[[\text{while } c \text{ do } s]]R = C[[\neg c]]I$

Itération concrète :
 $\text{lfp } F = \bigcup F^n(\emptyset)$
exact mais souvent infini.

Itération abstraite :
remplace ensembles par domaine abstrait.
Peut encore ne pas terminer.

Widening ∇ :
accélère la convergence.
En intervalles : borne instable $\rightarrow \pm\infty$.
Sûr, termine, mais imprécis.

Widening avec seuils :
passe par constantes utiles avant $\pm\infty$.

Widening retardé :
fait quelques tours normaux avant ∇ .

Déroulement de boucle :
analyse les N premiers tours séparément.

Narrowing Δ :
raffine après widening.
Exemple $[0, +\infty] \rightarrow [0, 51]$.

Lien eBPF :
join + widening évitent l'explosion des chemins avec boucles.

Lien eBPF / réunion

C'est le pont direct avec PREVAİL : au lieu d'énumérer tous les chemins eBPF, l'analyse fusionne les chemins avec join et calcule les invariants avec widening.

● Cours 11 - Produits réduits, domaines disjonctifs

Gagner en précision par combinaison de domaines et séparation de cas.

Résumé ultra court

Cours 11 = précision modulaire.

Produit simple :

$D\# = D1\# \times D2\#$

deux domaines en parallèle.

Produit réduit :

produit simple + réduction p .

Les domaines se communiquent leurs infos.

Exemple :

$([1,9], \text{pair}) \rightarrow ([2,8], \text{pair})$

Attention :

ne pas réduire après widening.

Domaines disjonctifs :

garder plusieurs cas au lieu de fusionner.

Pourquoi :

les domaines convexes perdent les trous.

$[-20,-10] \cup [10,20]$ devient $[-20,20]$

donc fausse alarme division par zéro.

Complétion par parties finies :

ensemble fini d'éléments abstraits.

précis mais peut exploser.

Partitionnement d'états :

un domaine par case de partition.

Partitionnement de traces :

distinguer les chemins d'exécution.

très précis mais risque d'explosion.

Lien eBPF :

utile pour comprendre combinaison tags + numérique + mémoire.

mais le papier évite trop de path-sensitivity pour ne pas exploser.

Lien eBPF / réunion

L'état abstrait eBPF combine déjà plusieurs dimensions : tags, valeurs numériques, cellules mémoire valides, octets numériques. C'est une logique de produit de domaines.

● Cours 12 - Domaines relationnels

Représenter des relations entre variables : zones, octogones, polyèdres.

Résumé ultra court

Cours 12 = domaines numériques relationnels.

Non-relationnel :

$X \mapsto [0,10], Y \mapsto [0,10]$

oublie $X \leq Y$, $X = Y+1$, etc.

Relationnel :

représente des contraintes entre variables.

Motivation :

parfois l'assertion finale est simple,
mais il faut une relation intermédiaire pour la prouver.

Polyèdres :

$\sum a_i V_i \leq c$

très expressif, très coûteux.

Zones :

$V_j - V_i \leq c$

contraintes de différences.

Représentation par graphe de potentiel ou DBM.

Coût $O(n^2)$ mémoire, $O(n^3)$ opérations.

Très utile pour offsets, boucles, tableaux.

Octogones :

$\pm V_i \pm V_j \leq c$

plus expressif que zones.

Plus lourd.

DBM :

$m[i][j] = c$ signifie $V_j - V_i \leq c$.

Closure :

calcule contraintes implicites par plus courts chemins.

Bottom :

cycle négatif = contraintes contradictoires.

Lien eBPF :

prouver $r1 + \text{offset} + \text{size} \leq \text{data_end}$

revient à garder des contraintes $X - Y \leq c$.

Donc zones = choix naturel.

Lien eBPF / réunion

Cours prioritaire pour eBPF : les zones expriment précisément les contraintes utiles sur offsets et data_end, avec un coût inférieur aux polyèdres.

● Cours 13 - Pointeurs et tableaux

Analyser la mémoire indirecte : pointeurs, alias, strong/weak update, tableaux.

Résumé ultra court

Cours 13 = pointeurs et tableaux.

Pourquoi pointeurs :
ignorer les pointeurs rend l'analyse non sûre.
*p peut modifier x sans nommer x.

Analyse de pointeurs :
vers quoi p peut-il pointer ?

Analyse d'alias :
p et q peuvent-ils pointer vers la même case ?

Langage :
 $X \leftarrow \text{alloc}()$
 $X \leftarrow \&Y$
 $X \leftarrow *Y$
 $*X \leftarrow Y$

Mémoire :
adresses = variables + heap
valeurs = entiers + adresses

Allocation site :
toutes les allocations d'une même ligne sont résumées par un site μ .

Strong update :
si cible unique, on remplace.

Weak update :
si cible possible multiple, on ajoute/joind les possibilités.

Andersen :
inclusion, plus précis.

Steensgaard :
unification, plus rapide.

Tableaux :
résumer toutes les cases par $A\#$.
 $A[i] \leftarrow e$ = weak update si i inconnu.

Expand/fold :
expand crée une copie abstraite d'une case.
fold replie la copie dans le résumé global.

Lien eBPF :
registres contiennent num ou pointeurs tagués.
régions connues : ctx, stack, packet, map.
il faut suivre offsets + stack spilling.

Lien eBPF / réunion

Cours prioritaire pour eBPF : registres tagués, régions connues, offsets, stack spilling et accès mémoire indirects.

● Cours 14 - Application industrielle : Astrée

Méthodologie industrielle : spécialiser l'analyseur à une famille de programmes.

Résumé ultra court

Cours 14 = Astrée, analyse industrielle du C avionique.

Objectif :
prouver absence d'erreurs à l'exécution.

Cible :
C embarqué critique
pas malloc, pas récursion
grande boucle synchrone
beaucoup de flottants
structures simples

Méthode :
spécialiser l'analyse à une famille de programmes.

Architecture :
front-end C
IR typé
simplification
analyse de dépendances
interprète abstrait

Domaines :
intervalles
octogones
congruences
BDD / arbres booléens
ellipsoïdes
filtres
partitionnement de traces
domaines mémoire/pointeurs

Idée clé :
beaucoup de petits domaines spécialisés + réductions.

Recette :
partir intervalles
analyser fausses alarmes
ajuster paramètres
améliorer domaines
ajouter réductions
ajouter domaines spécialisés

Lien eBPF :
PREVAIL fait pareil :
analyse spécialisée pour bytecode eBPF
tags + mémoire + zones
au lieu d'un analyseur universel.

Lien eBPF / réunion

Cours à lire en diagonale : il donne la méthode générale 'analyse spécialisée + domaines spécialisés', très utile pour expliquer un projet eBPF crédible.

Synthèse finale pour la réunion

Ce que tu dois pouvoir dire sans hésiter :

1. eBPF est un bon cas d'analyse statique sûre : bytecode non fiable exécuté dans le noyau.
2. Les valeurs eBPF ont un tag : num, inv, ctx, stk, pkt, map/shared.
3. Les pointeurs se modélisent comme (region, offset).
4. Les propriétés à prouver sont surtout : accès mémoire dans les bornes et pas de lecture non initialisée.
5. Les intervalles seuls sont trop faibles car il faut des relations entre registres/offsets.
6. Les zones sont naturelles : elles expriment $X - Y \leq c$.
7. Les boucles demandent des invariants calculés par point fixe, avec join et widening.
8. L'état abstrait combine plusieurs infos : tags, domaine numérique, mémoire, cellules valides.
9. Comme Astrée, un analyseur eBPF doit être spécialisé à une famille de programmes.
10. Pour un stage court : viser un sous-langage eBPF + une propriété mémoire + expérimentation.

Ordre de révision express

Temps	Action
1h30-2h	Cours 12 : zones / DBM / relationnel
1h30-2h	Cours 13 : pointeurs / mémoire / stack
1h	Cours 10 : boucles / widening
45 min	Cours 9 : abstraction / sûreté
30 min	Cours 14 : méthodologie industrielle
30 min	Cours 11 : produits réduits / disjonctions